UNIT IV – MULTIMODAL GENERATIVE AI APPLICATIONS

QUESTION 7 – ENGINEERING DOCUMENT SUMMARIZER

AI-Based Engineering Document Summarization

Professional Implementation Report

Question	Develop an AI application that summarizes a lengthy engineering document into a short and meaningful summary using a pre-trained language model.
Folder Name	Q7_Engineering_Document_Summarizer
Python File	document_summarizer.py
Input	Engineering PDF document
AI Service	Hugging Face Inference API with a pre-trained summarization model

1. AIM

To develop an AI application that accepts a lengthy engineering document, extracts its text, processes the document in manageable sections, and generates a concise and meaningful summary using a pre-trained language model.

2. OBJECTIVES

- Accept a lengthy engineering PDF through a Streamlit interface.
- Extract readable text from the uploaded document.
- Divide the extracted document into smaller sections suitable for model input.
- Generate summaries for the individual sections using a pre-trained summarization model.
- Combine the section summaries into a concise final summary.
- Display document statistics and summarization compression.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- PyPDF
- Hugging Face Hub / Inference API
- Pre-trained summarization model
- Internet connection for model inference
- Computer or laptop

4. FOLDER AND FILE STRUCTURE

```
UNIT_IV_Multimodal_Generative_AI
└── Q7_Engineering_Document_Summarizer
    └── document_summarizer.py
```

5. TECHNOLOGIES USED

Technology	Purpose	Role in Application
Python	Programming	Implements the summarization application
Streamlit	Web interface	Provides PDF upload and summary display
PyPDF	PDF processing	Extracts text from PDF pages
Hugging Face Inference API	AI inference	Provides access to the pre-trained summarization model
Pre-trained summarization model	Text generation	Converts long document sections into concise summaries

6. SYSTEM WORKFLOW

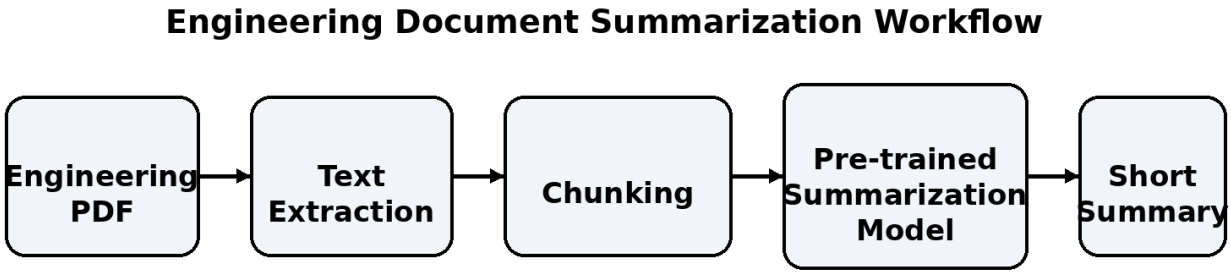

Figure 1. Engineering document summarization workflow.

The application accepts an engineering PDF, extracts its text, divides the document into manageable sections, sends each section to the pre-trained summarization model, combines the generated summaries, and displays the final concise summary.

7. STEP-BY-STEP IMPLEMENTATION

Step 1: Create the Q7_Engineering_Document_Summarizer folder.

Step 2: Create the document_summarizer.py file.

Step 3: Install Streamlit, Hugging Face Hub and PyPDF.

Step 4: Configure the Hugging Face access token using HF_TOKEN.

Step 5: Initialize the Hugging Face InferenceClient.

Step 6: Create a PDF uploader in Streamlit.

Step 7: Extract text page-by-page using PyPDF.

Step 8: Clean unnecessary whitespace from the extracted text.

Step 9: Divide the document into manageable chunks of approximately 2,500 characters.

Step 10: Summarize each chunk using the pre-trained summarization model.

Step 11: Combine the individual summaries.

Step 12: Perform a second summarization pass if the combined summary is still long.

Step 13: Display the final summary and document statistics.

Step 14: Calculate the percentage of the original document retained in the summary.

8. WHY CHUNKING IS USED

Long engineering documents may exceed the input capacity of a summarization model when sent as a single request. Therefore, the extracted document is divided into smaller sections before summarization. This chunk-based approach allows the application to process longer documents reliably and then combine the section summaries into one meaningful final summary.

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st
from huggingface_hub import InferenceClient
from pypdf import PdfReader
import os
import re

st.set_page_config(
    page_title="Engineering Document Summarizer",
    page_icon="📄",
    layout="wide"
)

st.title("📄 Engineering Document Summarizer")

st.write(
    "Upload a lengthy engineering document and generate "
    "a short and meaningful summary using a pre-trained "
    "language model."
)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:
    st.error("HF_TOKEN is not configured.")
    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

uploaded_file = st.file_uploader(
    "Upload a PDF engineering document",
    type=["pdf"]
)

def extract_text(pdf_file):
    reader = PdfReader(pdf_file)
    pages = []
    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:
            pages.append(page_text.strip())
    return "\n\n".join(pages)

def clean_text(text):
    text = re.sub(r"\s+", " ", text)
    return text.strip()

def split_into_chunks(text, max_chars=2500):
    words = text.split()
    chunks = []
    current_chunk = []
```

```
    current_length = 0

    for word in words:
        word_length = len(word) + 1

        if current_length + word_length > max_chars and current_chunk:
            chunks.append(" ".join(current_chunk))
            current_chunk = []
            current_length = 0

        current_chunk.append(word)
        current_length += word_length

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks

def summarize_chunk(text):
    result = client.summarization(
        text=text,
        model="facebook/bart-large-cnn"
    )
    return result.summary_text

def summarize_document(text):
    chunks = split_into_chunks(text, max_chars=2500)
    summaries = []

    for index, chunk in enumerate(chunks):
        summary = summarize_chunk(chunk)
        if summary.strip():
            summaries.append(summary.strip())

    combined_summary = " ".join(summaries)

    if len(combined_summary) <= 3500:
        return combined_summary

    final_chunks = split_into_chunks(
        combined_summary, max_chars=2500
    )

    final_summaries = []

    for chunk in final_chunks:
        final_summary = summarize_chunk(chunk)
        if final_summary.strip():
            final_summaries.append(final_summary.strip())

    return " ".join(final_summaries)
```

```
if uploaded_file is not None:

    st.success(f"Uploaded: {uploaded_file.name}")

    if st.button("📝 Generate Summary"):

        document_text = clean_text(
            extract_text(uploaded_file)
        )

        if not document_text:
            st.warning(
                "No readable text was found in the PDF."
            )
            st.stop()

        word_count = len(document_text.split())
        char_count = len(document_text)

        st.info(
            f"Extracted {word_count:,} words "
            f"and {char_count:,} characters."
        )

        chunks = split_into_chunks(
            document_text,
            max_chars=2500
        )

        st.write(
            f"Document divided into "
            f"**{len(chunks)} manageable sections**."
        )

        with st.spinner(
            "Generating meaningful engineering summary..."
        ):
            summary = summarize_document(
                document_text
            )

        st.subheader("📌 Generated Engineering Summary")
        st.success(summary)

        summary_words = len(summary.split())
        summary_chars = len(summary)

        st.subheader("📊 Document Statistics")

        col1, col2, col3 = st.columns(3)
```

```
with col1:
    st.metric(
        "Original Words",
        f"{word_count:,}"
    )

with col2:
    st.metric(
        "Summary Words",
        f"{summary_words:,}"
    )

with col3:
    st.metric(
        "Summary Characters",
        f"{summary_chars:,}"
    )

if word_count > 0:
    compression = (
        summary_words / word_count
    ) * 100

    reduction = 100 - compression

    st.subheader(
        "📉 Summarization Analysis"
    )

    st.write(
        f"Summary contains approximately "
        f"**{compression:.1f}%** of the original word count."
    )

    st.write(
        f"Approximately **{reduction:.1f}%** "
        f"of the original word count was removed."
    )
```

10. EXECUTION COMMAND

```
streamlit run ".\Q7_Engineering_Document_Summarizer\document_summarizer.py"
```

11. TEST DOCUMENT

The test document used for the implementation was an engineering technical document titled “Engineering_Operating_Systems_Technical_Document.pdf”. It contains engineering-oriented material on operating systems and their role in engineering computing.

12. IMPLEMENTATION RESULTS AND SCREENSHOTS

The screenshots below show the actual successful execution of the application, including PDF upload, text extraction, chunking, generated engineering summary, and summarization statistics.

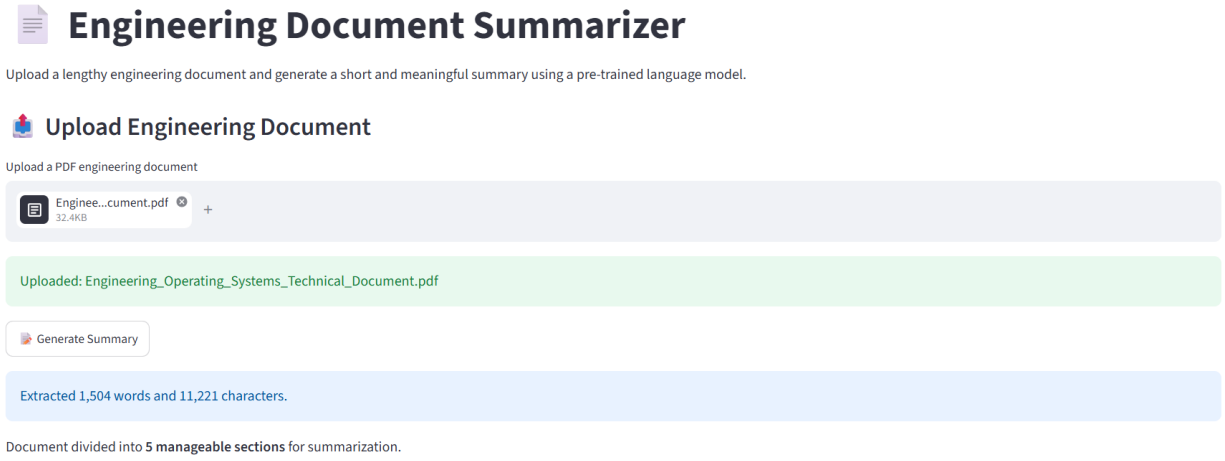

Figure 2. Uploaded engineering PDF, extracted text statistics and document chunking result.

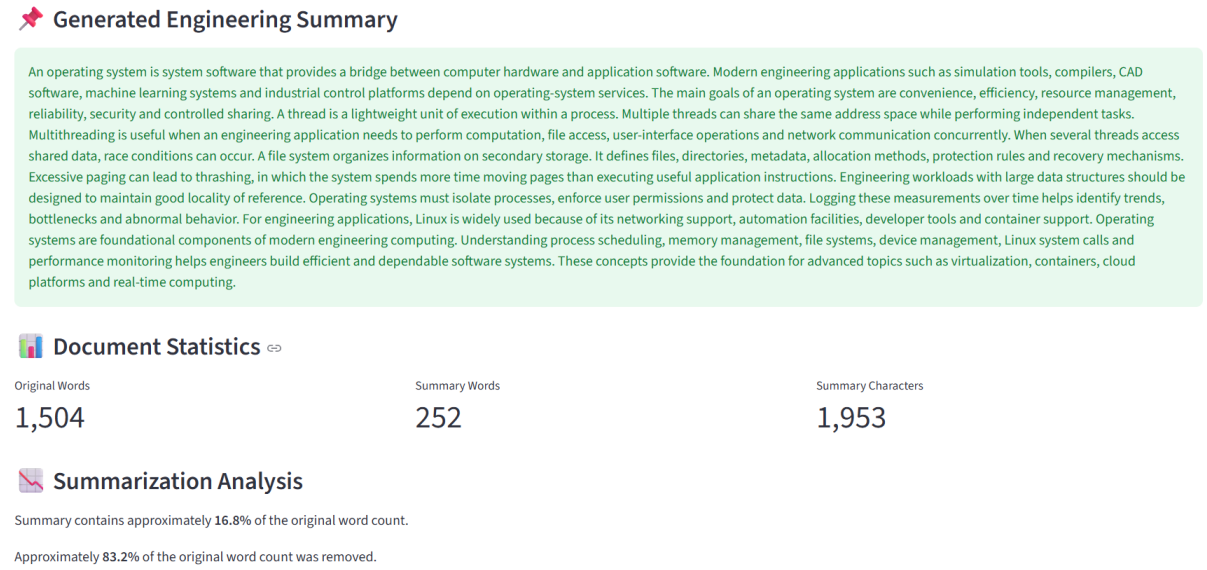

Figure 3. Generated engineering summary with document statistics and summarization analysis.

13. OBSERVED TEST RESULTS

Metric	Observed Value	Meaning
Original Words	1,504	Approximate number of extracted words in the uploaded engineering document.
Original Characters	11,221	Extracted document character count.
Document Sections	5	Number of manageable sections created for summarization.

Summary Words	252	Length of the final generated summary.
Summary Characters	1,953	Character count of the final summary.
Summary Retained	16.8%	Approximate fraction of original word count retained.
Word Reduction	83.2%	Approximate fraction of original word count removed.

14. SUMMARY QUALITY

The generated summary captures the central role of operating systems in engineering computing, including process and thread management, file systems, memory management, security, Linux usage and performance monitoring. The summary is substantially shorter than the source while retaining the major concepts needed for a concise technical overview.

15. ADVANTAGES

- Can summarize long engineering documents automatically.
- Uses a pre-trained language model, so no model training is required.
- Chunking allows longer documents to be processed more reliably.
- Provides a concise summary that is easier for students to review.
- Displays useful statistics to show the reduction in document length.

16. LIMITATIONS

- PDFs containing scanned images may require OCR before text extraction.
- Very complex technical content may lose some detail during summarization.
- Summary quality depends on the pre-trained model and extracted text quality.
- API access requires internet connectivity.

17. FUTURE ENHANCEMENTS

- Support DOCX, TXT and web documents in addition to PDF.
- Add OCR for scanned engineering documents.
- Allow the user to select summary length.
- Generate bullet-point summaries and key-term lists.
- Add multilingual summarization for engineering documents.
- Provide page references for important summarized concepts.

18. RESULT

The Engineering Document Summarizer was successfully implemented and tested. The uploaded PDF contained approximately 1,504 words and was divided into five manageable sections. The system generated a 252-word summary, retaining approximately 16.8% of the original word count and removing approximately 83.2%. This demonstrates successful automatic compression of a lengthy engineering document into a concise technical summary.

19. CONCLUSION

The developed application demonstrates a practical use of Generative AI for engineering education. By combining PDF text extraction, chunk-based processing and a pre-trained summarization model, the system converts lengthy technical material into a shorter and meaningful form. The successful test results confirm that the application satisfies the requirement of summarizing an engineering document using a pre-trained language model.

20. VIVA / KEY POINTS

- Text summarization automatically reduces a long document into a shorter form while preserving important information.
- PyPDF is used to extract text from the uploaded PDF.
- Chunking is used because a long document may exceed the model's input length.
- A pre-trained summarization model is used to generate concise summaries.
- Streamlit provides the user interface for document upload and output display.